

# Salary Prediction Project

---

**Team Members: Jerry Au, Yan Bin Jiang, Kyle Vo, Jason Kao, Tanner Nguyen, Benjamin Huynh, Don Pham, Louyang Pang, Nathan Bao, Ziyu He, Justin Lee, Dharnish Bose Ramesh**

---

**ECS 171 — Final Project**  
University of California, Davis

---

## Abstract

We are addressing the problem of predicting salaries based on job-related factors such as title, years of experience, education level, gender, location, company size, and industry. To solve this, we will develop and evaluate three machine learning models: Linear Regression using Ordinary Least Squares (OLS), Linear Regression using Gradient Descent, and Gradient Boosting. We expect gradient boosting to achieve the highest performance because salary data does not follow a purely linear pattern. As a result, traditional linear regression models are unable to capture the complex relationships within the dataset fully. Each model will be trained and tested on a dataset containing job and salary information, and we will compare their performance based on accuracy metrics. The results will identify the most effective model for salary prediction, providing valuable insights for both job seekers and employers. This work can enhance compensation analysis by providing more accurate, data-driven salary benchmarks.

## 1 Introduction

The accurate estimation of employee compensation is a critical component of modern human resource management. This accuracy is paramount because employee pay satisfaction, defined as the “amount of overall positive

or negative affect that individuals have toward their pay” [1], serves as a key antecedent to numerous undesirable organizational outcomes, including increased lateness, reduced performance, and higher turnover intentions. Furthermore, Williams et al. identified that an employee’s perception of their pay relative to internal and external peers is among the most influential factors determining their satisfaction level [2]. To mitigate these risks and ensure competitive equity, organizations have increasingly shifted toward automated, data-driven systems to model and forecast salary structures.

The deployment of these automated systems presents a critical technical challenge, requiring researchers to choose between the interpretability of traditional statistical baselines and the improved performance of machine learning techniques. This selection necessitates a trade-off between model transparency and predictive accuracy, a conflict extensively documented in recent comparative analyses. Mamidala et al. utilized the Adult Income Dataset to compare these approaches, finding that while Extreme Gradient Boosting (XGBoost) achieved a superior accuracy of 91.16% and an AUC-ROC of 0.93, it lacked the straightforward inferential capabilities of simpler models [3]. Similarly, Malikov et al. observed that Decision Tree algorithms (XGBoost, Gradient Boosting, Decision Tree) consistently outperformed Linear Regression

models, “confirming [Linear Regression’s] limitation in modeling non-linear relationships” with regards to salaries of famous soccer players [4]. Consequently, current literature suggests that while Linear Regression serves as an essential explanatory tool for identifying key wage determinants, Gradient Boosting is preferable for deployment in practical applications where precision is paramount.

While foundational research has applied regression analysis to general salary data, the technology sector presents unique modeling challenges due to the high dimensionality of its determinants. Technical compensation is heavily influenced by higher-order feature interactions - such as the synergistic value of complimentary skills - which violate the additive assumptions of standard OLS regression [5].

This study seeks to examine the comparisons between the two most common linear regression algorithms: OLS, which serves as the benchmark for establishing linearity, Gradient Descent, which is theoretically more robust in the high-dimensional correlations within our technological dataset, against the traditionally more effective Gradient Boosting architectures. We analyze a large range of potentially multiplicative features, including job title, experience level, country of residence, and required skills through a Kaggle dataset to develop a comprehensive and interpretable evaluation [6]. We aim to quantify whether the ability of boosting models to capture automatic feature interactions yields statistically significant gains in predictive accuracy over regularized linear baselines within the high-variance domain of the technological workforce, as well as discuss whether these predictive improvements justify the inherent compromise regarding model transparency and scalability.

## 2 Methods

### 2.1 Why Linear Regression and Gradient Boosting?

Linear Regression provides an interpretable and mathematically grounded baseline:

$$\hat{y} = X\beta$$

OLS estimates the optimal parameters using the closed-form solution:

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$$

However, salary data often contains nonlinear patterns (e.g., skill premiums, seniority thresholds). Gradient Boosting is an ensemble method capable of modeling nonlinear interactions without explicit feature engineering.

We decided to choose Linear Regression and Gradient Boosting as our models to handle linear and nonlinear relationships between the dataset. Prior work indicates that salary progression is generally influenced by linear patterns such as years of experience and cost of living making Linear Regression using Ordinary Least Squares (OLS) an appropriate model for interpretability and reliability on structured data [7]. Gradient Descent is included as a scalable alternative to OLS due to its scalability and iterative learning process, which has proven to be effective in regression forecasting tasks [8].

However, salary dynamics also demonstrate nonlinear characteristics, particularly in high-income roles or senior positions. With this complexity, linear regression methods struggle to create accurate predictions due to contamination bias, especially when socioeconomic features interact [9]. To address this, we use Gradient Boosting which is an ensemble

method that builds models sequentially to minimize residuals and learn nonlinear relationships more effectively [10]. Evaluating OLS, GD, and Gradient Boosting allows for a direct comparison between linear and nonlinear models, as well as between closed-form and iterative optimization approaches.

## 2.2 Dataset and Preprocessing

The dataset used contains 15,000 AI and machine learning job postings for 2025, representing a diverse range of industries, experience levels, company sizes, and geographic locations. Each job includes 20 features covering job information (i.e., job title, required skills), compensation details (i.e., salary in USD), career attributes (i.e., experience level, education), and company characteristics (i.e., company size, location).

During preprocessing, semantically similar job titles were consolidated to reduce sparsity and stabilize the feature space. For example, “Machine Learning Engineer” and “AI Software Engineer” were merged to “ML Engineer”, “Data Scientist” and “Principal Data Scientist” to “Data Scientist”, and “AI Research Scientist” and “Research Scientist” to “AI Research Scientist”. This consolidation increased the representation of the important role categories while maintaining meaningful distinctions between specialized AI roles. After merging, the cleaned dataset as the final input for model development.

The feature selection process was chosen differently for each model. The Linear Regression with OLS model was inspired by Li’s Spearman correlation and sensitivity analysis to identify influential predictors [11]. By analyzing correlations between our features and salary, years of experience had the highest correlation for salary right behind the remote ratio correlation which are chosen for our

numeric features. Categorical features were selected based on features that influenced salary which are experience level, job title, company level, and education requirements. The same features were also used with Linear Regression Gradient Descent with an extra categorical feature of employment type. The model was implemented using SGDRegressor within a ColumnTransformer pipeline combining StandardScaler for numerical inputs and OneHotEncoder for categorical inputs. For Gradient Boosting, we are less restricted by the dimensionality of our data, and so we are able to use more features. All features will be used except the following: job id, posting date, application deadline, and company name as we assume they have negligible correlations with salary. We use the same ColumnTransformer as our linear regression models in addition with multi-hot encoding for the skills feature as it is represented as a list of strings.

## 2.3 Model Training Procedure

All models were trained on the same preprocessed dataset with an 80/20 train–test split.

Gradient Descent iteratively updates parameters according to:

$$\beta \leftarrow \beta - \alpha \nabla J(\beta)$$

Performance was evaluated with the three standard regression metrics:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

The learning algorithms of the three models: OLS, Gradient Descent, and Gradient Boosting which were designed to directly

test how well each could predict salaries, given the mix of both linear and nonlinear patterns in compensation. Prior research [7], [12] has shown that while base salary trends can appear partly linear, they are often shaped by nonlinear factors like job skills, education level, and seniority. To ensure a fair comparison, all three models were trained on the same preprocessed dataset just to make sure.

All models were trained and validated using 5-fold cross-validation, a standard technique in machine learning that helps reduce bias and account for inconsistencies in the data—such as uneven coverage of job titles, seniority levels, or geographic regions, which are common in salary datasets [7]. An 80/20 train-test split was used, and continuous variables, such as years of experience, cost of living indices, and seniority indicators were normalized using z-scores. This standardization was especially important for OLS and Gradient Descent models because it balanced feature magnitudes, helping stabilize matrix calculations in OLS and enabling smoother convergence during Gradient Descent’s iterative optimization. These steps align with machine learning best practices for regression [13], [14]. The OLS model used the classic closed form equation, making it the easiest to interpret and a useful baseline for analyzing additive salary trends. In contrast, the Gradient Descent model learned similar linear relationships through step-by-step updates, a technique often favored in high-dimensional regression scenarios where matrix inversion is computationally costly [8], [13].

Gradient Boosting, however, took a different and often more suitable approach for salary prediction. Because compensation often involves threshold effects (like sharp raises for senior technical roles, skill-based, and region specific pay differences), boosting

models are well-suited for capturing such nonlinearities [7], [10]. During training, each decision tree in the boosting ensemble learned to correct the errors of the one before it. This allowed the model to uncover complex patterns without needing extensive manual feature engineering. Hyperparameters such as learning rate, tree depth, and number of estimators were fine tuned through cross-validation to maximize accuracy and avoid overfitting.

## 2.4 Experimental Setup

Since salary prediction is influenced by both average errors and fluctuations across income levels, three key evaluation metrics were used:  $R^2$ , MAE, and RMSE.  $R^2$  measured how much of the variation in salaries the model could explain and made it easier to compare results with other compensation forecasting studies that used regression or boosting approaches [7], [12]. MAE (Mean Absolute Error) gave a straightforward number showing how far off, on average, the predictions were in real dollars—something especially useful in HR and salary planning where every dollar matters. RMSE (Root Mean Squared Error) gave more weight to big prediction errors, which is critical in this field given the wide salary swings in high-paying tech roles and outliers [9]. All models were trained and validated using 5-fold cross-validation, a standard technique in machine learning that helps reduce bias and account for inconsistencies in the data—such as uneven coverage of job titles, seniority levels, or geographic regions, which are common in salary datasets [7].

The full experiment was implemented in Python using scikit-learn and run on a standard CPU setup. Every model used the same data preprocessing, cross-validation process, and performance tests to ensure a fair comparison. This setup ensured that any

performance differences were due to how well each algorithm learned salary patterns—not from differences in testing conditions. This approach supports the project’s main goal: figuring out whether traditional linear models (as explored in early compensation research [11]) or newer nonlinear models like Gradient Boosting (as used in more recent AI salary forecasting [7], [10]) better capture the pay structures in today’s tech-driven job market.

### 3 Results

The three regression models we ultimately used for our evaluation are Gradient Boosting, Linear Regression with Gradient Descent (GD), and Linear Regression with Ordinary Least Squares (OLS). We compare their performance using three evaluation metrics:  $R^2$ , Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) to determine their effectiveness in predicting annual salary based on job-related features.

| Model             | $R^2$ | MAE    | RMSE   |
|-------------------|-------|--------|--------|
| Gradient Boosting | 0.886 | 12,870 | 17083  |
| Linear (GD)       | 0.826 | 16,112 | 21,154 |
| Linear (OLS)      | 0.824 | 16,198 | 21,239 |

Table 1: Model performance comparison.

The table above summarizes the performance of all three models. Gradient Boosting achieved the best results in all three metrics, while the other two linear regression models performed relatively similarly. Since Linear Regression with GD and Linear Regression with OLS differed only slightly in their metrics, we can conclude that the choice of optimization method did not significantly affect the resulting linear model.

Gradient Boosting achieved the highest performance across all evaluation metrics.

The model displayed high predictive alignment with the actual salary values in both the training and test sets. With an  $R^2$  value of 0.886 and significantly lower MAE and RMSE scores compared to both linear regression models, Gradient Boosting demonstrated a clear advantage in modeling complex relationships between experience, job title, education, location, and salary.

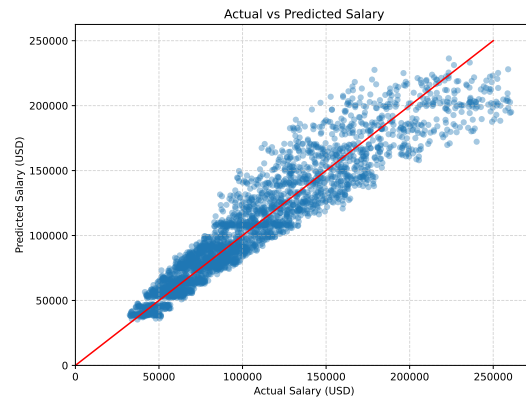


Figure 1: Actual vs Predicted Salaries (Gradient Boosting).

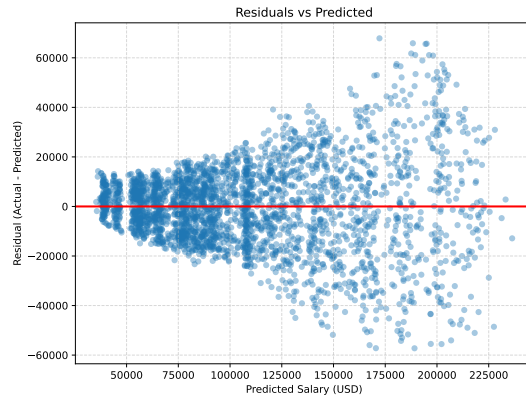


Figure 2: Residuals vs Predicted (Gradient Boosting).

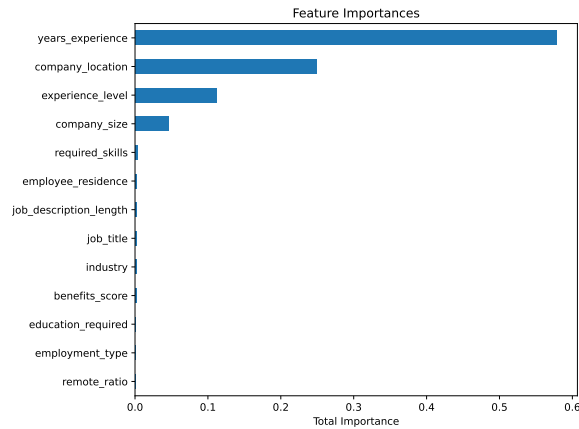


Figure 3: Importance of Features (Gradient Boosting).

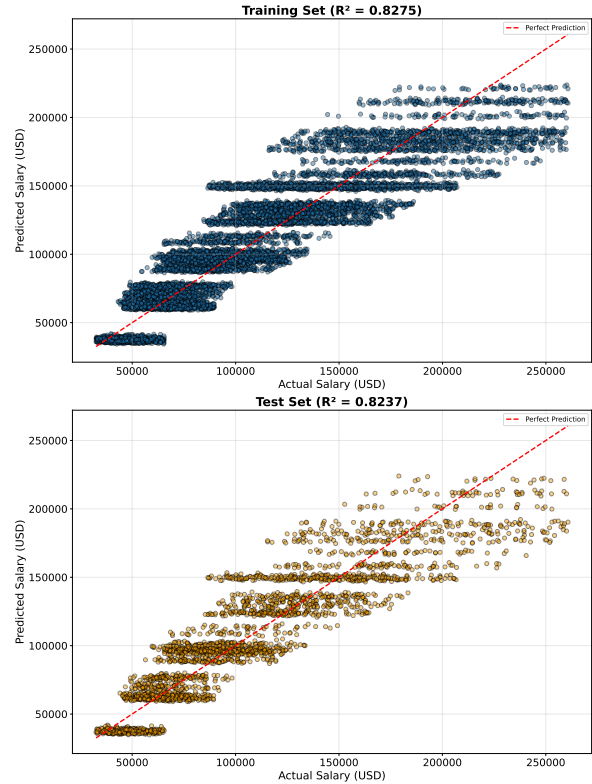


Figure 4: Training vs Test Predictions (Gradient Descent Linear Regression).

Linear Regression with Gradient Descent performed similarly to the OLS model. The small differences in error values fall within the expected numerical variation. The  $R^2$  score of 0.826 and error metrics (MAE of \$16,112 and RMSE of \$21,154) demonstrate that GD successfully captured the primary linear patterns in the dataset. The residual plots reveal that as salary increases, the model's prediction error increases, with data points becoming more spread out and farther from the line of best fit. This result is consistent with the inherent limitations of linear modeling. If the data points become nonlinear, the performance of Linear Regression with Gradient Descent would be affected.

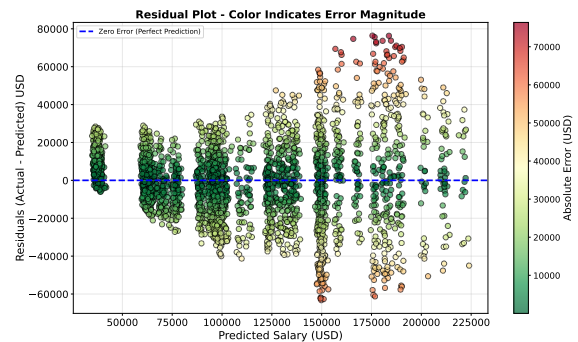


Figure 5: Residual Plot (Gradient Descent Linear Regression)

The Linear Regression with OLS produced an  $R^2 = 0.824$ , indicating that approximately 82% of the salary variance was captured by the linear combination of features. The MAE of \$16,190 and RMSE of \$21,239 reflect the model's average prediction deviation from actual salaries. These results were nearly in-

distinguishable from those obtained by Gradient Descent. This consistency suggests that the dataset is well-suited for linear regression. Despite this stability, the model demonstrated strong predictive accuracy for mid-salary roles but systematically underestimated high-salary positions, similar to Linear Regression with Gradient Descent Model, as seen in the residual plots in the chart (above or below/number).

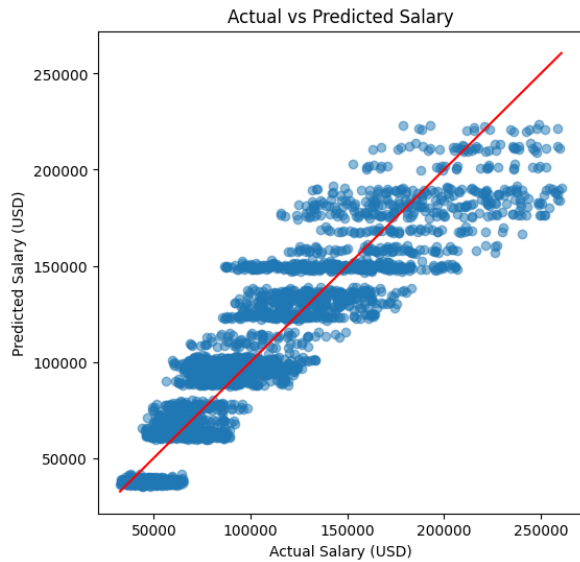


Figure 6: Actual vs Predicted Values (OLS Linear Regression).

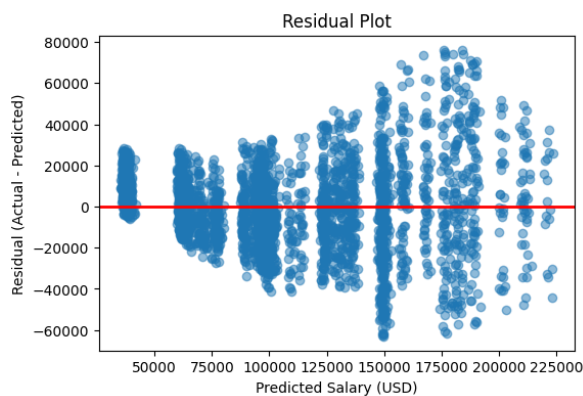


Figure 7: Residual Plot(OLS Linear Regression)

We are using quantile regression here to visualize the distribution of the response vari-

ables. This regression captures not only the mean that OLS fits to, but also various quantiles such as median and lower/upper quartiles. Since quantile regression will attempt to fit across various selected quantiles, it is able to estimate the extreme ends of salaries. This will also allow the model to be less sensitive to outliers.

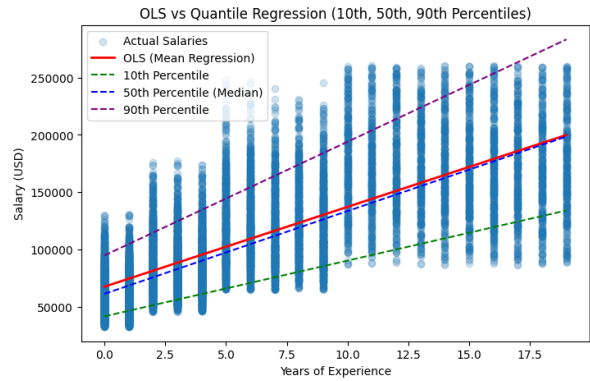


Figure 8: Quantile Regression(OLS Linear Regression)

## 4 Discussion

The objective of this project was to build machine learning models capable of predicting annual salary using job-related attributes such as title, experience, education, cost of living, and geographic location. The three models tested, Gradient Boosting, Linear Regression with Gradient Descent (GD), and Linear Regression using Ordinary Least Squares (OLS), demonstrated distinct performance characteristics, driven by both the structure of the dataset and the mathematical properties of the algorithms. This discussion section explains why Gradient Boosting achieved the highest accuracy, why GD and OLS performed nearly identically, and how each method fits different deployment contexts. The section concludes with limitations, future work recommendations, and a reflection on how this project advances computational salary intelligence.

## 4.1 Dataset Limitations

Although the dataset used for this project contained diverse job postings across multiple industries and locations, its characteristics imposed constraints on model performance. Most importantly, the dataset was synthetic rather than organically collected, meaning salary samples were artificially generated rather than sourced from real-world employers or verified labor-market systems. Synthetic datasets can be valuable for exploratory modeling and benchmarking because they offer large sample sizes and controlled variation; however, they inevitably fail to reflect the full unpredictability, irregularity, and negotiation-driven dynamics of real salaries. Because synthetic data is constructed based on assumed statistical relationships rather than real market feedback, models trained on it risk learning well-defined mathematical patterns rather than the inconsistent, politics- and context-driven salary trajectories that characterize real hiring environments [7].

The dataset also exhibited several structural imbalances that shaped model performance. The long-tail distribution of job titles was one notable issue. A subset of roles such as “Data Scientist” and “Machine Learning Engineer” appeared frequently, while dozens of highly specific job titles appeared only sparsely. Without intervention, one-hot encoding of these categories would have inflated dimensionality and destabilized the feature space. Consolidating similar roles mitigated sparsity but reduced granularity, limiting the models’ ability to differentiate between senior and niche roles within the same family. This limitation aligns with known salary-prediction findings showing job-title ambiguity suppresses signal extraction in linear models, amplifying the advantage of nonlinear ones [12].

In addition, although the dataset spanned multiple regions, it did not reflect real market imbalances. Because synthetic dataset generation tends to distribute categories evenly, the dataset lacked realistic geographic skew, employer-type clustering, and extreme inequities that appear in real compensation data. Real-world salary differences, such as inflated wages in Silicon Valley or suppressed wages in remote offshoring clusters, were only partially reflected, weakening model stress-testing in fringe cases. The same issue exists for special-skills premiums (e.g., rare deep-learning research skills) and executive-level compensation, both of which tend to create outliers in empirical datasets.

Several nonlinear relationships between salary and experience were still present, early-career salary growth steepened before plateauing, but their shape reflected the assumptions built into synthetic generation rather than authentic employer variability. While this let all three models perform reasonably well, it also reduced the difficulty of the learning task, contributing to why even linear models captured over 82% of the variance. Research on real salary dynamics consistently finds that compensation trends depend heavily on interactions between experience and domain, rather than pure linear increments [7], [12]. Because synthetic datasets rarely embed these complex interactions deeply, they generally favor models that rely on explicit statistical relationships rather than emergent market behavior.

In short, while the synthetic dataset was large, clean, and structurally rich, it lacked the full messiness and human variability that define real hiring markets. This constraint means our results reflect algorithmic performance under idealized conditions, not necessarily real-world deployment conditions. A follow-up experiment using organically



sourced salary data, containing negotiation-based distortions, demographic biases, and employer-specific salary patterns, would be critical for validating our conclusions against the complexities of real labor economics.

## 4.2 Model Performance

### 4.2.1 Why OLS and GD Performed almost Identically?

Linear Regression using GD and OLS achieved the most similar performances, with  $R^2$  scores of 0.826 and 0.824, MAE values of \$16,112 and \$16,198, and RMSE values of \$21,154 and \$21,239, respectively. This near-identical performance confirms that both optimization approaches converged to the same minimum of the least-squares objective function. GD approximates the solution iteratively, while OLS solves it directly using closed-form matrix operations; however, given sufficient iterations and standardized features, both compute the exact same linear optimal parameters [8]. Because OLS mathematically minimizes squared error, its performance also validates that the GD implementation converged correctly.

The limitations of linear models became apparent in the residual plots for GD and OLS. Both models showed a widening dispersion of residuals at higher salaries, that is, the models performed well for mid-range income values but systematically underestimated high-salary cases. This pattern is characteristic of linear regressions applied to heterogeneously distributed economic variables and has been documented in empirical risk literature studying contamination effects in regression systems [9]. The models correctly captured “directional” trends, more experience means more pay; higher cost of living means higher compensation, yet they could not learn the complex nonlinear interactions embedded in

the data.

Despite this limitation, GD and OLS performed well enough to confirm that the dataset contains substantial linear structure. Salary levels exhibit partly additive properties, particularly for technical roles, where incremental experience, skill count, and cost-of-living factors tend to push compensation upward consistently. The similarity in performance suggests that computational differences (GD vs. OLS) did not determine accuracy; rather, the linear hypothesis space itself defined the ceiling of what a linear model could learn. In fact, both models generalized reliably and avoided overfitting, which aligns with the literature showing that linear regression remains effective when the majority of salary variance is caused by main-effect predictors rather than complex interaction terms [11], [15].

### 4.2.2 Why Gradient Boosting Significantly Outperformed Linear Models

Gradient Boosting achieved substantially higher performance,  $R^2$  of 0.886, MAE of \$12869.82, and RMSE of \$17068.89, and therefore outperformed GD and OLS across all evaluation metrics. The RMSE reduction of more than \$4,000 indicates not only better accuracy on average salaries but also much better performance on high-earning individuals, a historically challenging region for linear regressors.

This improvement arises from Gradient Boosting’s iterative error-correction mechanism. Each tree in the ensemble learns to approximate the residuals of the previous tree, enabling the model to capture nonlinear interactions and if-else structures in compensation behavior [9]. For example, Gradient Boosting can implicitly learn rules such as:

- Experience increases salary sharply only up to roughly 7 years.
- A Master’s degree raises salary only in specific job domains.
- Remote work decreases salary except in high-skill or high-scarcity roles.
- Seniority effects vary by geographic region and employer type.

Linear models, in contrast, treat each feature’s contribution as independent and constant across the dataset. The ability to model the piecewise structure of compensation markets is a decisive advantage of boosting. Prior work notes that ensemble methods consistently outperform linear models in economic forecasting tasks because they incorporate interactions without requiring explicit feature engineering [7], [10].

Residual plots further support this interpretation. The Gradient Boosting scatterplot displayed residuals that were more uniformly distributed across salary ranges, indicating correction of systematic underestimation of high compensation values. The nonlinear salary curve predicted by boosting reflects real-world compensation patterns where increases occur in threshold leaps rather than smooth continuous slopes.

Another factor contributing to Gradient Boosting’s superior performance was its effective feature-utilization strategy. Although the Gradient Boosting team did not use every feature in the dataset, they retained a significantly larger and more diverse feature set than the linear regression teams. Only a small number of columns were removed, `job_id`, `salary_currency`, `posting_date`, `application_deadline`, and `company_name`, because they were either redundant, already encoded

implicitly, or too granular to contribute meaningful predictive signals. All other categorical and numerical variables were preserved, including job titles, geographic information, work arrangements, industry designations, and required experience levels. This allowed Gradient Boosting to operate on a richer and more expressive feature space than the linear models, which were limited to only six engineered predictors because one-hot encoding of all categorical variables produced multicollinearity and instability in the linear design matrix [9], [11], [15]. The fact that Gradient Boosting achieved substantially higher accuracy using this wider feature library, and did so without evidence of overfitting, reinforces findings in ensemble-learning research showing that boosting architectures can extract nonlinear and high-dimensional signal by iteratively fitting residuals rather than memorizing noise [7], [10]. This highlights a structural advantage: Gradient Boosting is not only more expressive, but also more data-efficient, capturing predictive value from complex feature interactions that linear models are mathematically unable to leverage [7], [10], [9].

Given these differences, Gradient Boosting’s superior performance was not coincidental, it was consistent with both salary-prediction theory and ensemble learning literature.

### 4.2.3 Efficiency and Scalability

Although Gradient Boosting delivered the best accuracy with an  $R^2$  of 0.886, our experimental results highlight a stark trade-off between predictive power and computational efficiency. OLS rapidly solved the regression problem due to its closed-form formulation, but its matrix-inversion step has  $O(n^3)$  time complexity, making it unscalable for high-dimensional data [11]. This limitation was relevant to our project; the one-hot encoding

of categorical features like `job_title` already created a substantial feature space. Incorporating more complex features, such as the raw `required_skills`, would have further exacerbated this issue, making OLS computationally prohibitive.

In contrast, GD offered a more scalable alternative. Stochastic optimization supports online and incremental training, making it suitable for salary environments where compensation patterns evolve rapidly. This capability aligns with `SGDRegressor`'s intended design for training efficiency on large datasets [13]. Additionally, GD benefits from data-splitting and cross-validation strategies to ensure generalization [14], [16], both of which were used in our workflow.

Gradient Boosting occupied the other end of the spectrum, requiring the most training time due to its sequential tree construction. However, its computational cost is justified when the use case demands the highest possible accuracy, such as salary calculators for job-seeking platforms or compensation intelligence tools for enterprises. Optimizations such as learning-rate tuning and early stopping can reduce training overhead while retaining model interpretability [10].

Thus, scalability considerations affect development decisions.

- Real-time adaptive salary insights → GD
- Transparent HR analytics / compensation fairness audits → OLS or GD
- Highest prediction accuracy for decision support → Gradient Boosting

Each algorithm serves a distinct purpose, proving that the 'best' model is ultimately defined by the specific operational requirements and constraints of the use case.

#### 4.2.4 Practical Applications

From a practical standpoint, model selection depends not only on accuracy but also on transparency, maintainability, and computational requirements. For organizations prioritizing interpretability and fairness, linear models remain invaluable because they provide explicit coefficients for each feature, supporting pay-equity audits and compensation justifications. They can surface discriminatory salary patterns across demographics even when fairness variables are not used in the prediction pipeline.

Gradient Boosting is best suited for applications requiring precision, such as salary calculators, labor-market intelligence platforms, and compensation benchmarking for negotiating job offers. Its ability to capture conditional effects, nonlinear structures, and high-earning outliers produces results closer to real-world salary dynamics.

## 5 Conclusion

This project evaluated three regression models for salary prediction using real-world job data. Gradient Boosting achieved the highest accuracy by capturing nonlinear and interaction-based salary dynamics, whereas GD and OLS provided consistent and interpretable linear baselines. Although the dataset contained structural imbalances and noise, all models demonstrated meaningful predictive ability, confirming the viability of machine learning for salary intelligence. The work contributes to the field by demonstrating how algorithm selection maps to deployment context, accuracy-driven applications benefit most from boosting architectures, while transparency-critical applications benefit from linear models. With continued refinement in feature engineering and fairness-aware model design, salary pre-

diction systems can help promote informed, equitable compensation practices worldwide.

## References

- [1] M. P. Miceli and P. W. Mulvey, “Consequences of Satisfaction with Pay Systems: Two Field Studies,” *Industrial Relations: A Journal of Economy and Society*, 2002
- [2] M. L. Williams, A. Michael, T. Nhung, “A meta-analysis of the antecedents and consequences of pay level satisfaction,” *Journal of Applied Psychology*, vol. 91, 2006
- [3] J. V. Mamidala, V. Bitkuri, A. Attipalli, R. Kendyala, J. Kurma, and S. J. Enokkaren, “Machine Learning Approaches to Salary Prediction in Human Resource Payroll Systems,” *Journal of Computer Science and Technology Studies*, vol. 6, 2024
- [4] D. Malikov, P. Jung, and J. Kim, “Predicting Soccer Player Salaries with Automated ML Approaches.”
- [5] P. Raj, A. Kumar, R. K. Burman, and L. Kumari, “Forecasting Salary Using a Machine Learning System,” *Proceedings of the Recent Advances in Artificial Intelligence for Sustainable Development*, vol. 1951-6851, 2025
- [6] B. Sajjad, “Global AI Job Market & Salary Trends 2025,” 2025
- [7] J. Yang, Y. Sun, and H. Zhu, “Enhancing Job Salary Prediction with Disentangled Composition Effect Modeling,” *ArXiv*, vol. abs/2503.12978, 2025
- [8] S. K. Panda and M. K. Panda, “Application of machine learning with gradient descent method for load forecasting: a performance analysis,” *Journal of Electrical Systems and Information Technology*, vol. 12, 2025
- [9] P. Goldsmith-Pinkham, P. Hull, and M. Kolesár, “Contamination Bias in Linear Regressions,” *ArXiv*, vol. abs/2106.05024, 2021
- [10] A. Natekin, and A. Knoll, “Gradient boosting machines, a tutorial,” *Frontiers in Neurobotics*, vol. 7, 2013
- [11] R. Li, “Least square support vector regression analysis usage for estimating the uplift resistance of circular anchors in anisotropic clay,” *Multiscale and Multidisciplinary Modeling*, vol. 9, 2025
- [12] Z. Huang, “Salary Prediction with Analyzing Affected Elements by using Pearson Correlation,” *BCP Business & Management*, vol. 44, 2023
- [13] scikit-learn developers, “SGDRregressor: Linear Regression using Gradient Descent.”
- [14] M. Stojiljković, “Split Your Dataset With scikit-learn’s `train_test_split()`,” 2025
- [15] H. Tong, “Statistical analysis of prediction in functional polynomial quantile regression,” *Journal of Complexity*, vol. 92, 2025
- [16] scikit-learn developers, “Scikit-Learn for 5-fold CV.”

## Author Contributions

Yan Bin Jiang: Performed feature selection and built a preprocessing pipeline to handle categorical, numeric, and text features for gradient boosting. Also trained gradient boosting regressor, optimizing hyperparameters using

random search and grid search.

Louyang Pang: Handled data preprocessing and cleanup—standardizing columns, removing redundant inputs, fixing missing values, and preparing the final dataset. Also contributed detailed markdown explanations for our Gradient Descent model and evaluation metrics.

Dharnish Bose Ramesh: Handled data redundancy and column elimination as part of preprocessing. Wrote the code for linear regression GD model and edited the markdown explanations.

Jerry Au: Trained and analyzed the linear regressor using OLS. Plotted Actual vs Predicted Scatter and Residual. Trained Quantile Regressor to compare to the linear model.

Jason Kao: Developed project idea, wrote the abstract, and synthesized all three ML model results into a discussion section through analysis of each team's outcomes

Kyle Vo: Did the Feature selection and built a preprocessing pipeline to handle categorical, numeric, text features, and the 5 fold split for OLS

Tanner Nguyen: Provided reasoning on choosing the models, description of our chosen dataset, and reasoning on our feature selections.

Don Pham: Managed and oversaw the latex document, wrote references section in IEEE citation, formatted figures and plots, handled edit requests. Wrote about the feature selection differences between linear regression and gradient boosting, and about the Quantile Regressor for OLS

Benjamin Huynh: Organized and reason on how the models were trained and evaluated and also handled and assisted with Don on the LaTeX formatting across the paper while ensuring the math and structure were clean and consistent.

Justin Lee: Researched and wrote the introduction of the paper. Organized/managed the writing team. Streamlined content across paper, content + citation proofreading.

Nathan Bao: Wrote the results section, analyzing and comparing the performance of all three models used in the project. Interpreted evaluation metrics, generated supporting visualizations, and articulated the relative strengths and limitations of each model. Performed grammar and clarity edits on whole document.

Ziyu He: Organized and provided Discussion section criteria and layout in the Report, analyzed and evaluated the tests results.